

Top-down Parsing

a.y. 2025-2026

1 / 76

Parsing

- Given a grammar $\mathcal{G} = (V, T, S, \mathcal{P})$ and a word w
- Say whether $w \in \mathcal{L}(\mathcal{G})$ and, if so, provide its derivation tree

2 / 76

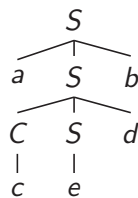
Parsing

- Two relevant kinds of parsing
 - Top-down: construct leftmost derivation from the root to the yield
 - Bottom-up: construct rightmost derivation (in reverse order) from the yield to the root

3 / 76

Parsing

TOP-DOWN

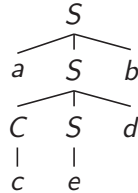


- Top-down: Take, in the given order:
 - $S \rightarrow aSb$
 - $S \rightarrow CSd$
 - $C \rightarrow c$
 - $S \rightarrow e$

4 / 76

Parsing

BOTTOM-UP



- Bottom-up: Take, in reverse order:
 - $C \rightarrow c$
 - $S \rightarrow e$
 - $S \rightarrow CSd$
 - $S \rightarrow aSb$

5 / 76

Top-down parsing

- Let $w = bd$ and

$$\begin{aligned} \mathcal{G}: S &\rightarrow Ad \mid Bd \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

- Let us try to derive w
- If we choose $S \Rightarrow Ad$ we fail to get bd
- If we choose $S \Rightarrow Bd$ we get it through the derivation $S \Rightarrow Bd \Rightarrow bd$

6 / 76

Top-down parsing

- Let us try with another example now
- Let $w = id + id * id$ and

$$\begin{aligned} \mathcal{G} : \quad E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

- What is, if any, a leftmost derivation of w in $\mathcal{G}$?

7 / 76

Top-down parsing

- Let $w = cad$ and

$$\begin{aligned} \mathcal{G} : \quad S &\rightarrow cAd \\ A &\rightarrow ab \mid a \end{aligned}$$

- What is a leftmost derivation of w in $\mathcal{G}$?
- At the first step we get $S \Rightarrow cAd$
- By choosing $A \rightarrow ab$ we get $S \Rightarrow cabd$
- Bad luck, backtrack

8 / 76

Predictive top-down parsing

- To avoid backtrack we focus on *predictive top-down parsing* which successfully applies to a specific sub-class of free grammars
- LL(1) grammars
- For such grammars we can deterministically provide a *leftmost* derivation of the given string (LL(1)) by reading the string from the *left* (LL(1)) *one* symbol at a time (LL(**1**))

9 / 76

Predictive top-down parsing

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

For LL(1) grammars we can set up a parsing table that can drive leftmost derivations

(Symbol \$ is used as right-delimiter of input strings)

	<i>id</i>	+	*	(	)	\$
<i>E</i>	$E \rightarrow TE'$			$E \rightarrow TE'$		
<i>E'</i>		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
<i>T</i>	$T \rightarrow FT'$			$T \rightarrow FT'$		
<i>T'</i>		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
<i>F</i>	$F \rightarrow id$			$F \rightarrow (E)$		

10 / 76

Predictive top-down parsing

- We have a word, say $id + id * id$, and we must derive it from E
- Check the table entry $[E, id]$
- The entry contains $E \rightarrow TE'$, take the derivation step $E \Rightarrow TE'$

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

11 / 76

Predictive top-down parsing

- The input is $id + id * id$ and the current derivation is TE'
- The leftmost non-terminal is T , the current symbol is id , then check the entry $[T, id]$
- The entry contains $T \rightarrow FT'$, take the derivation step $TE' \Rightarrow FT'E'$

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

12 / 76

Predictive top-down parsing

- The input is still $id + id * id$ and the current derivation is FTE'
- The leftmost non-terminal is F , the current symbol is id , then check the entry $[F, id]$
- The entry contains $F \rightarrow id$, take the derivation step $FT'E' \Rightarrow idT'E'$
- The first occurrence of id in $id + id * id$ matches with the first symbol of $idT'E'$, move the input pointer forward to point to '+' and go on checking the entry $[T', +]$

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

13 / 76

Parsing tables

How do we fill the entries of parsing tables?

The entry $M[A, b]$ is consulted to expand A when the next input character is b

Then we set $M[A, b] = A \rightarrow \alpha$ if

- Either it is possible to derive from α a string starting with b
 - Namely, $\alpha \Rightarrow^* b\beta$ for some β
- Or α can derive ϵ and in some derivation A is followed by some γ from which it is possible to derive a string starting with b
 - Namely, $\alpha \Rightarrow^* \epsilon$ and $S \Rightarrow^* wA\gamma$ for some w and γ with $\gamma \Rightarrow^* b\beta$ for some β

14 / 76

Parsing tables

Take the grammar

$$\begin{aligned} S &\rightarrow aA \mid bB \\ A &\rightarrow c \\ B &\rightarrow c \end{aligned}$$

How would you fill the table for it?

	<i>a</i>	<i>b</i>	<i>c</i>	\$
<i>S</i>				
<i>A</i>				
<i>B</i>				

Parsing tables

Take the grammar

$$\begin{aligned} S &\rightarrow aAb \\ A &\rightarrow \epsilon \end{aligned}$$

How would you fill the table for it?

	<i>a</i>	<i>b</i>	\$
<i>S</i>			
<i>A</i>			

$first(\alpha)$

Set of terminals that begin strings derived from α

Moreover, if $\alpha \Rightarrow^* \epsilon$ then $\epsilon \in first(\alpha)$

$first(\alpha)$

$first(\epsilon) = \{\epsilon\}$

$first(a) = \{a\}$

$first(A) = \bigcup_{A \rightarrow \alpha} first(\alpha)$

Computation of $first(\alpha)$ for $\alpha = Y_1 \dots Y_n$:

```
first( $Y_1 \dots Y_n$ ) =  $\emptyset$ ;  
 $j = 1$  ;  
while  $j \leq n$  do  
    add  $first(Y_j) \setminus \{\epsilon\}$  to  $first(Y_1 \dots Y_n)$  ;  
    if  $\epsilon \in first(Y_j)$  then  $j = j + 1$  ;  
    else break ;  
if  $j = n + 1$  then add  $\epsilon$  to  $first(Y_1 \dots Y_n)$  ;
```

$first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

19 / 76

 $first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Consider the productions for F , compute $first(F)$

	elements in $first$
E	
E'	
T	
T'	
F	(, id

20 / 76

$first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Consider the productions for T' , compute $first(T')$

	elements in $first$
E	
E'	
T	
T'	$*, \epsilon$
F	$(, id$

21 / 76

 $first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Consider the production for T , compute $first(T)$

	elements in $first$
E	
E'	
T	$(, id$
T'	$*, \epsilon$
F	$(, id$

22 / 76

$first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Consider the productions for E' , compute $first(E')$

	elements in $first$
E	
E'	$+, \epsilon$
T	$(, id$
T'	$*, \epsilon$
F	$(, id$

23 / 76

 $first(\alpha)$

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Consider the production for E , compute $first(E)$

	elements in $first$
E	$(, id$
E'	$+, \epsilon$
T	$(, id$
T'	$*, \epsilon$
F	$(, id$

24 / 76

$\text{follow}(A)$

- $\text{follow}(A)$ is the set of terminals that can follow A in some derivation

25 / 76

$\text{follow}(A)$

```
follow( $S$ ) = { $\$$ };  
foreach  $A \neq S$  do  
   $\text{follow}(A) = \emptyset$ ;  
repeat  
  foreach  $B \rightarrow \alpha A \beta$  do  
    if  $\beta \neq \epsilon$  then  
       $\text{add first}(\beta) \setminus \{\epsilon\}$  to  $\text{follow}(A)$   
    if  $\beta = \epsilon$  or  $\epsilon \in \text{first}(\beta)$  then  
       $\text{add follow}(B)$  to  $\text{follow}(A)$   
until saturation;
```

26 / 76

follow(A)

TRAINING

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon \\
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

27 / 76

follow(A)

TRAINING

```

for  $B \rightarrow \alpha A \beta$ 
if  $\beta \neq \epsilon$  then add  $\text{first}(\beta) \setminus \{\epsilon\}$  to  $\text{follow}(A)$ 
if  $\beta = \epsilon$  or  $\epsilon \in \text{first}(\beta)$  then add  $\text{follow}(B)$  to  $\text{follow}(A)$ 

```

$$E \rightarrow TE'$$

- First **if-then** ● Add $\text{first}(E') \setminus \{\epsilon\}$ to $\text{follow}(T)$, hence add $+$ to $\text{follow}(T)$
- Second **if-then** ● (Case $\beta = \epsilon$) Add $\text{follow}(E)$ to $\text{follow}(E')$, keep the information
- (Case $\epsilon \in \text{first}(\beta)$) Add $\text{follow}(E)$ to $\text{follow}(T)$, keep the information

28 / 76

$follow(A)$

TRAINING

```

for  $B \rightarrow \alpha A \beta$ 
if  $\beta \neq \epsilon$  then add  $first(\beta) \setminus \{\epsilon\}$  to  $follow(A)$ 
if  $\beta = \epsilon$  or  $\epsilon \in first(\beta)$  then add  $follow(B)$  to  $follow(A)$ 

```

$$E' \rightarrow +TE'$$

- First **if-then** • Add $first(E') \setminus \{\epsilon\}$ to $follow(T)$, hence add $+$ to $follow(T)$
- Second **if-then** • (Case $\beta = \epsilon$) Add $follow(E')$ to $follow(E')$, nothing to do
- (Case $\epsilon \in first(\beta)$) Add $follow(E')$ to $follow(T)$, keep the information

 $follow(A)$

TRAINING

```

for  $B \rightarrow \alpha A \beta$ 
if  $\beta \neq \epsilon$  then add  $first(\beta) \setminus \{\epsilon\}$  to  $follow(A)$ 
if  $\beta = \epsilon$  or  $\epsilon \in first(\beta)$  then add  $follow(B)$  to  $follow(A)$ 

```

$$T \rightarrow FT'$$

- First **if-then** • Add $first(T') \setminus \{\epsilon\}$ to $follow(F)$, hence add $*$ to $follow(F)$
- Second **if-then** • (Case $\beta = \epsilon$) Add $follow(T)$ to $follow(T')$, keep the information
- (Case $\epsilon \in first(\beta)$) Add $follow(T)$ to $follow(F)$, keep the information

$follow(A)$

TRAINING

```

for  $B \rightarrow \alpha A \beta$ 
if  $\beta \neq \epsilon$  then add  $first(\beta) \setminus \{\epsilon\}$  to  $follow(A)$ 
if  $\beta = \epsilon$  or  $\epsilon \in first(\beta)$  then add  $follow(B)$  to  $follow(A)$ 

```

$$T' \rightarrow *FT'$$

- First **if-then** • Add $first(T') \setminus \{\epsilon\}$ to $follow(F)$, hence add $*$ to $follow(F)$
- Second **if-then** • (Case $\beta = \epsilon$) Add $follow(T')$ to $follow(T')$, nothing to do
- (Case $\epsilon \in first(\beta)$) Add $follow(T')$ to $follow(F)$, keep the information

 $follow(A)$

TRAINING

```

for  $B \rightarrow \alpha A \beta$ 
if  $\beta \neq \epsilon$  then add  $first(\beta) \setminus \{\epsilon\}$  to  $follow(A)$ 
if  $\beta = \epsilon$  or  $\epsilon \in first(\beta)$  then add  $follow(B)$  to  $follow(A)$ 

```

$$F \rightarrow (E)$$

- First **if-then** • Add $first() \setminus \{\epsilon\}$ to $follow(E)$, hence add $') '$ to $follow(E)$
- Second **if-then** • (Case $\beta = \epsilon$) Not applicable
- (Case $\epsilon \in first(\beta)$) Not applicable

$follow(A)$

TRAINING

Now add \$ to $follow(E)$ and elaborate the information kept aside

$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$	<pre> foreach $B \rightarrow \alpha A \beta$ do if $\beta \neq \epsilon$ then add $first(\beta) \setminus \{\epsilon\}$ to $follow(A)$ end if $\beta = \epsilon$ or $\epsilon \in first(\beta)$ then add $follow(B)$ to $follow(A)$ end end </pre>
--	--

	first	computation of follow	follow
E	{id, (}	\$;)	{\$,)}
E'	{ ϵ , +}	add follow(E)	{\$,)}
T	{id, (}	+; add follow(E); +; add follow(E')	{\$,), +}
T'	{ ϵ , *}	add follow(T)	{\$,), +}
F	{id, (}	*; add follow(T); *; add follow(T')	{\$,), +, *}

33 / 76

 $follow(A)$

TRAINING

$$\begin{aligned} S &\rightarrow aABb \\ A &\rightarrow Ac \mid d \\ B &\rightarrow CD \\ C &\rightarrow e \mid \epsilon \\ D &\rightarrow f \mid \epsilon \end{aligned}$$

34 / 76

$follow(A)$

TRAINING

$S \rightarrow aABb$
 $A \rightarrow Ac \mid d$
 $B \rightarrow CD$
 $C \rightarrow e \mid \epsilon$
 $D \rightarrow f \mid \epsilon$

```

foreach  $B \rightarrow \alpha A \beta$  do
  if  $\beta \neq \epsilon$  then
    | add  $first(\beta) \setminus \{\epsilon\}$  to  $follow(A)$ 
  end
  if  $\beta = \epsilon$  or  $\epsilon \in first(\beta)$  then
    | add  $follow(B)$  to  $follow(A)$ 
  end
end

```

	first	computation of follow	follow
S	a	\$	\$
A	d	(production 1) $\beta = Bb$, add e f b (production 2) $\beta = c$, add c	b c e f
B	e, epsilon	(production 1) $\beta = b$, add b	b
C	e, epsilon	(production 4) $\beta = D$, add f	b f
D	f, epsilon	(production 4) $\beta = \epsilon$	b

35 / 76

 $follow(A)$

TRAINING

$S \rightarrow aA \mid bBc$
 $A \rightarrow Bd \mid Cc$
 $B \rightarrow e \mid \epsilon$
 $C \rightarrow f \mid \epsilon$

36 / 76

$\text{follow}(A)$

TRAINING

$S \rightarrow aA \mid bBc$
 $A \rightarrow Bd \mid Cc$
 $B \rightarrow e \mid \epsilon$
 $C \rightarrow f \mid \epsilon$

```

foreach  $B \rightarrow \alpha A \beta$  do
  if  $\beta \neq \epsilon$  then
    | add  $\text{first}(\beta) \setminus \{\epsilon\}$  to  $\text{follow}(A)$ 
  end
  if  $\beta = \epsilon$  or  $\epsilon \in \text{first}(\beta)$  then
    | add  $\text{follow}(B)$  to  $\text{follow}(A)$ 
  end
end

```

	first	computation of follow	follow
S	a, b	\$	\$
A	e, d, f, c	(production 1) $\text{beta}=\epsilon$	\$
B	e, epsilon	(production 2) $\text{beta}=c$, add c (production 3) $\text{beta}=d$, add d	c d
C	f, epsilon	(production 4) $\text{beta}=c$, add c	c

37 / 76

Construction of predictive parsing tables

ALGORITHM

```

input      : Grammar  $\mathcal{G} = (V, T, S, \mathcal{P})$ 
output     : Predictive parsing table  $M$ 
foreach  $A \rightarrow \alpha \in \mathcal{P}$  do
  | add  $A \rightarrow \alpha$  to  $M[A, b]$  for each  $b \in \text{first}(\alpha)$ 
  | if  $\epsilon \in \text{first}(\alpha)$  then
    | | add  $A \rightarrow \alpha$  to  $M[A, x]$  for each  $x \in \text{follow}(A)$ 
  |
set to error() all the empty entries;

```

Recall: by notational convention b is a terminal symbol

Observe: $\text{follow}(A)$ can contain ϵ , hence we use x (rather than b) to range over it

38 / 76

LL(1) grammars

A context-free grammar is **LL(1)** iff no entry of the predictive parsing table for $\mathcal{G}$ is multiply-defined

39 / 76

Algorithm for predictive top-down parsing

- **Input:**
string w ; top-down parsing table M for $\mathcal{G} = (V, T, S, \mathcal{P})$
- **Output:**
leftmost derivation of w if $w \in \mathcal{L}(\mathcal{G})$, error() otherwise
- **Initialization:**
 $w\$$ in the input buffer; $\$S$ onto the stack, with S on top

40 / 76

Algorithm for predictive top-down parsing

```

let  $b$  be the first symbol of  $w\$$  ;
let  $X$  be the top of the stack ;
while  $X \neq \$$  do
  if  $X = b$  then
    pop  $X$  ;
    let  $b$  be the next symbol of  $w\$$  ;
  else if  $X$  is a terminal then error();
  else if  $M[X, b]$  is error then error();
  else if  $M[X, b] = X \rightarrow Y_1 \dots Y_k$  then
    output( $X \rightarrow Y_1 \dots Y_k$ ) ;
    pop  $X$  ;
    push  $Y_k$  ; ... ; push  $Y_1$  ;
  let  $X$  be the top of the stack ;

```

41 / 76

Example of application

Parsing $id + id * id$ with the table

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

42 / 76

Example of application

stack	input	output	leftmost derivation
\$E	id + id * id\$	$E \rightarrow TE'$	$E \Rightarrow TE'$
\$E'T	id + id * id\$	$T \rightarrow FT'$	$\Rightarrow FT'E'$
\$E'T'F	id + id * id\$	$F \rightarrow id$	$\Rightarrow idT'E'$
\$E'T'id	id + id * id\$		
\$E'T'	+id * id\$	$T' \rightarrow \epsilon$	$\Rightarrow idE'$
\$E'	+id * id\$	$E' \rightarrow +TE'$	$\Rightarrow id+TE'$
\$E'T+	+id * id\$		
\$E'T	id * id\$	$T \rightarrow FT'$	$\Rightarrow id+FT'E'$
\$E'T'F	id * id\$	$F \rightarrow id$	$\Rightarrow id+idT'E'$
\$E'T'id	id * id\$		
\$E'T'	*id\$	$T' \rightarrow *FT'$	$\Rightarrow id+id*FT'E'$
\$E'T'F*	*id\$		
\$E'T'F	id\$	$F \rightarrow id$	$\Rightarrow id+id*idT'E'$
\$E'T'id	id\$		
\$E'T'	\$	$T' \rightarrow \epsilon$	$\Rightarrow id+id*idE'$
\$E'	\$	$E' \rightarrow \epsilon$	$\Rightarrow id+id*id$
\$	\$		

43 / 76

Left recursion

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

Is it LL(1)?

$$\text{first}(E) = \text{first}(T) = \text{first}(F) = \{ (, id \}$$

Hence, e.g.,

- $M[E, id]$ contains both $E \rightarrow E + T$ and $E \rightarrow T$

What if we always expand E into $E + T$ when we see the input id ?

$$E \Rightarrow E + T \Rightarrow E + T + T \Rightarrow E + T + T + T \dots$$

44 / 76

Left recursion

A grammar is **left recursive** if, for some A and some α , $A \Rightarrow^* A\alpha$

Example:

$$\begin{aligned} S &\rightarrow B \mid a \\ B &\rightarrow Sa \mid b \end{aligned}$$

Is left recursive by $S \Rightarrow B \Rightarrow Sa$

Left recursion

A grammar is **immediately left recursive** if it has a production of the form $A \rightarrow A\alpha$

Example:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

Is immediately left recursive by either $E \rightarrow E + T$ or $T \rightarrow T * F$

Left recursion

LEMMA

If $\mathcal{G}$ is left recursive then $\mathcal{G}$ is not LL(1)

47 / 76

Elimination of immediate left recursion

GOAL

Transform

$$A \rightarrow A\alpha \mid \beta \text{ where } \alpha \neq \epsilon \text{ and } \beta \neq A\gamma$$

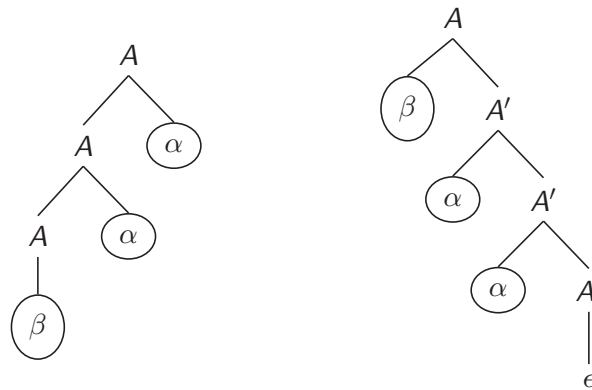
To get a non left recursive grammar which generates the same language

48 / 76

Elimination of immediate left recursion

INTUITION

$$A \rightarrow A\alpha \mid \beta \text{ where } \alpha \neq \epsilon \text{ and } \beta \neq A\gamma$$



49 / 76

Elimination of immediate left recursion

STRATEGY

Substitute

$$A \rightarrow A\alpha \mid \beta$$

Where $\alpha \neq \epsilon$ and $\beta \neq A\gamma$

With

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

Where A' is a fresh non-terminal

50 / 76

Elimination of immediate left recursion

STRATEGY FOR THE MOST GENERAL CASE

Substitute

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_k$$

Where $\alpha_j \neq \epsilon$ for every $j = 1 \dots n$ and $\beta_i \neq A\gamma_i$ for every $i = 1 \dots k$

With

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \dots \mid \beta_k A' \\ A' &\rightarrow \alpha_1 A' \mid \dots \mid \alpha_n A' \mid \epsilon \end{aligned}$$

Where A' is a fresh non-terminal

51 / 76

Elimination of left recursion

INTUITION

- Transform the grammar so to decrease the number of steps of the derivation $A \Rightarrow^* A\alpha$
- Eliminate immediate left recursion

52 / 76

Elimination of left recursion

EXAMPLE

Take

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow Bc \mid Ad \mid b \end{aligned}$$

Left recursion shows in two derivation steps:

$$A \Rightarrow Ba \Rightarrow Ada$$

To decrease the number of steps of the derivation, replace the production $B \rightarrow Ad$ by

$$B \rightarrow Bad \mid bd$$

Elimination of left recursion

EXAMPLE

Get

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow Bc \mid Bad \mid bd \mid b \end{aligned}$$

Eliminate immediate left recursion from B and get the grammar

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow bdB' \mid bB' \\ B' &\rightarrow cB' \mid adB' \mid \epsilon \end{aligned}$$

Elimination of left recursion

EFFECTIVENESS

Eliminate left recursion from the grammar

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

Get the LL(1) grammar

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

55 / 76

Elimination of immediate left recursion

EFFECTIVENESS

Eliminate left recursion from the **ambiguous** grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

Get the grammar

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

56 / 76

Elimination of immediate left recursion

EFFECTIVENESS

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

Is it LL(1)?

Not really: the entry $[E', +]$ of the parsing table contains two productions:

- $E' \rightarrow +EE'$
- $E' \rightarrow \epsilon$

In fact:

	elements in first	elements in follow
E	(id	\$ + *)
E'	+ * ϵ	\$ + *)

Lesson learnt: Elimination of left recursion does not guarantee to get an LL(1) grammar

Elimination of immediate left recursion

EFFECTIVENESS

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

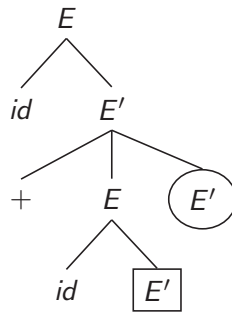
Is it ambiguous or not?

Elimination of immediate left recursion

EFFECTIVENESS

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

Take the partial derivation tree



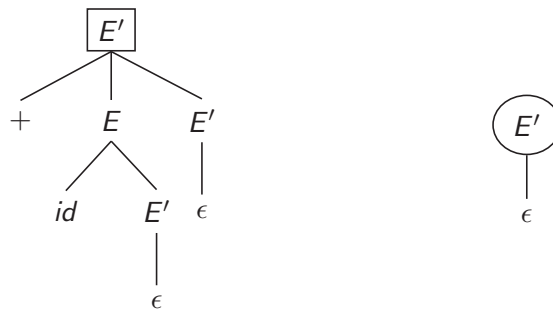
59 / 76

Elimination of immediate left recursion

EFFECTIVENESS

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

Get a derivation of $id + id + id$ by completing the tree with this instantiation of the missing sub-trees

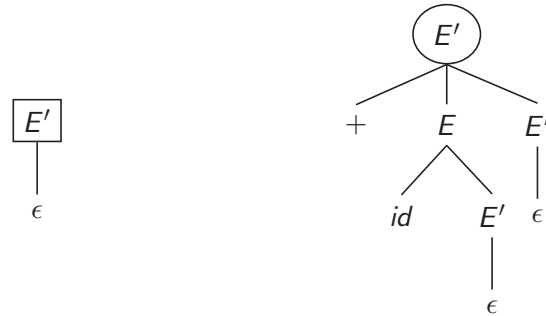


60 / 76

Elimination of immediate left recursion

EFFECTIVENESS

Get a different derivation of $id + id + id$ by completing the tree with this instantiation



Lesson learnt: Elimination of left recursion does not eliminate ambiguity

61 / 76

Left factoring

Take the grammar

$$S \rightarrow aSb \mid ab$$

Is it LL(1)?

Not really: the entry $[S, a]$ of the parsing table contains two productions:

- $S \rightarrow aSb$
- $S \rightarrow ab$

62 / 76

Left factoring

A grammar can be **left factorized** (can undergo left factorization) if multiple productions for the same non-terminal have the same prefix

LEMMA

If $\mathcal{G}$ can be left factorized then $\mathcal{G}$ is not LL(1)

63 / 76

Left factoring

STRATEGY

Delay as much as possible the choice between productions with the same prefix

Substitute

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

With

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

Where A' is a fresh non-terminal

64 / 76

Left factoring

ALGORITHM

```

input           : Grammar  $\mathcal{G}$  that can be left factorized
output          : Left factorized version of  $\mathcal{G}$ 
repeat
  foreach  $A$  do
    find the longest prefix  $\alpha$  common to two or more
    productions for  $A$  ;
    if  $\alpha \neq \epsilon$  then
      choose a fresh non-terminal  $A'$  and replace
       $A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \dots \mid \gamma_k$  by
       $A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_k$ 
       $A' \rightarrow \beta_1 \mid \dots \mid \beta_n$ 
  until no pair of productions for any  $A$  has common prefix;

```

65 / 76

Left factoring

EFFECTIVENESS

Apply left factorization to

$$S \rightarrow aSb \mid ab$$

Get the grammar

$$\begin{aligned} S &\rightarrow aS' \\ S' &\rightarrow Sb \mid b \end{aligned}$$

Is it LL(1)?

66 / 76

Left factoring

EFFECTIVENESS

$$\begin{aligned} S &\rightarrow aS' \\ S' &\rightarrow Sb \mid b \end{aligned}$$

	elements in first	elements in follow
S	a	$\$ b$
S'	$a b$	$\$ b$

Top-down parsing table

	a	b	$\$$
S	$S \rightarrow aS'$		
S'	$S' \rightarrow Sb$	$S' \rightarrow b$	

67 / 76

Left factoring

EFFECTIVENESS: DANGLING ELSE

Apply left factorization to the **ambiguous** grammar

$$S \rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } S \text{ else } S \mid c$$

Get the grammar

$$\begin{aligned} S &\rightarrow \text{if } b \text{ then } S S' \mid c \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

68 / 76

Left factoring

EFFECTIVENESS

$$\begin{aligned} S &\rightarrow \text{if } b \text{ then } S S' \mid c \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

Is it LL(1)?

Not really: the entry $[S', \text{else}]$ of the parsing table contains two productions:

- $S' \rightarrow \text{else } S$
- $S' \rightarrow \epsilon$

In fact:

	elements in first	elements in follow
S	if c	\$ else
S'	else ϵ	\$ else

Lesson learnt: Left factorization does not guarantee to get an LL(1) grammar

Left factoring

EFFECTIVENESS

$$\begin{aligned} S &\rightarrow \text{if } b \text{ then } S S' \mid c \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

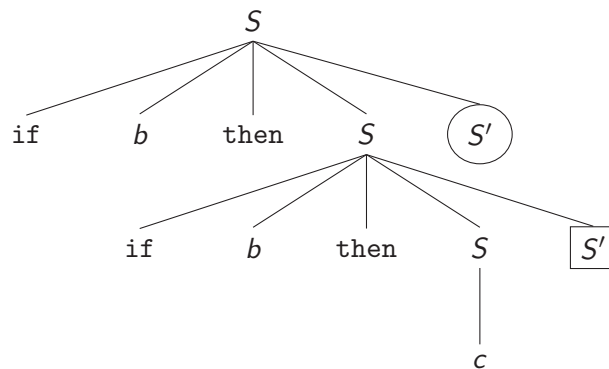
Is it ambiguous or not?

Left factoring

EFFECTIVENESS

$$\begin{aligned} S &\rightarrow \text{if } b \text{ then } S S' \mid c \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

Take the partial derivation tree



71 / 76

Left factoring

EFFECTIVENESS

$$\begin{aligned} S &\rightarrow \text{if } b \text{ then } S S' \mid c \\ S' &\rightarrow \text{else } S \mid \epsilon \end{aligned}$$

Get a derivation of “if b then if b then c else c ” by completing the tree with this instantiation of the missing sub-trees

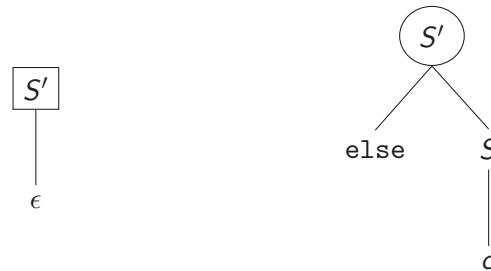


72 / 76

Left factoring

EFFECTIVENESS

Get a different derivation of “if b then if b then c else c ” by completing the tree with this instantiation



Lesson learnt: Left factorization does not eliminate ambiguity

73 / 76

Dangling else

INNERMOST BINDING

A common strategy to avoid the ambiguity due to the dangling else is to impose the so-called **innermost binding**: every else must match the closest unmatched then

Innermost binding can be enforced by defining a grammar that allows only matched then-else pairs between occurrences of then and else

$$\begin{aligned}
 S &\rightarrow M \mid U \\
 M &\rightarrow \text{if } b \text{ then } M \text{ else } M \mid c \\
 U &\rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } M \text{ else } U
 \end{aligned}$$

74 / 76

Summary

- No left-recursive grammar is LL(1)
- No grammar that can be left-factorized is LL(1)
- No ambiguous grammar is LL(1)

75 / 76

LL(1) grammars

LEMMA

$\mathcal{G}$ is LL(1) iff if $\mathcal{G}$ has productions $A \rightarrow \alpha \mid \beta$ then

- $\text{first}(\alpha) \cap \text{first}(\beta) = \emptyset$
- If $\epsilon \in \text{first}(\alpha)$ then $\text{first}(\beta) \cap \text{follow}(A) = \emptyset$ and, v.v., if $\epsilon \in \text{first}(\beta)$ then $\text{first}(\alpha) \cap \text{follow}(A) = \emptyset$

76 / 76